

Guličky a stroj

Paulínka sa rada hrá s guľičkami. A keďže je veľmi zručná, vytvorila špeciálny stroj, do ktorého ich môže vkladat.

Vnútroštruktúra tohto stroja vieme popísať nasledovne:

- Obsahuje N **vrcholov** očíslovaných od 0 po $N - 1$, pričom vrchol $N - 1$ voláme **koreň**.
- Každý vrchol u , pre $0 \leq u < N - 1$, má jedného **rodiča**. Rodičom vrcholu u je vrchol s číslom $P[u]$, pričom $P[u] > u$. Vrchol u potom voláme **dieťa** vrcholu $P[u]$. Koreň nemá žiadneho rodiča. Vrchol, ktorý nemá žiadne dieťa, voláme **list**.

Paulínka svoj stroj uzavrela do nepriehľadnej kovovej krabice, **neviete** preto ani hodnotu N (počet vrcholov) ani rodiča $P[u]$ pre žiaden vrchol u . Paulínka vám však prezradila, že jej stroj má M listov, ktoré sú očíslované 0 až $M - 1$. Vašou úlohou je pohrať sa so strojom a zistiť ako vyzerá.

Paulínka má fakt veľa guľičiek. Na každú z nich môžete napísať jedno nezáporné celé číslo. Toto číslo je **hodnotou** danej guľičky. V každom vrchole stroja môže byť **najviac jedna guľička**. Vrchol je **obsadený** ak je v ňom guľička, inak ho voláme **voľný**. Na začiatku sú všetky vrcholy voľné.

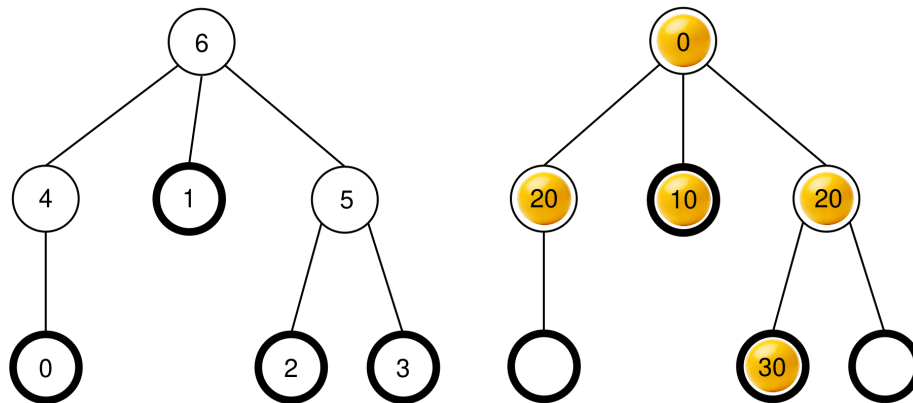
So strojom môžete robiť nasledovné dve operácie:

- **insert**(U, X): na novú guľičku napíšete hodnotu X a pokúsite sa ju vložiť do listu U .
 - Ak je list U obsadený, táto operácia vráti `false` a guľička do stroja vložená nebude.
 - Ak je list U voľný, guľička je doň vložená. Následne, ak je rodič vrchola, v ktorom je aktuálne guľička, voľný, presunie sa doňho. Toto posúvanie guľičky opakuje, až kým nenarazí na obsadeného rodiča alebo sa nebude nachádzať v koreni. Pri takomto úspešnom vložení guľičky vráti funkcia `true`.
- **collect**(): ak je koreň stroja voľný (v stroji sa teda nenachádza ani jedna guľička), vráti táto funkcia prázdne pole. Inak táto funkcia inicializuje pole S na prázdne pole a následne zavolá `traverse(N-1)`. Funkcia `traverse` je popísaná nižšie. Je to rekurzívna funkcia, ktorá postupne zozbiera hodnoty všetkých guľičiek v stroji do poľa S .

```
traverse(u):  
    pridaj hodnotu guľičky vo vrchole u na koniec poľa S  
    nech c[u] je zoznam všetkých obsadených detí vrcholu u  
    zorad' c[u] podľa hodnôt guľičiek v týchto listoch do neklesajúcej postupnosti  
    (ak viacero listov obsahuje guľičku s rovnakou hodnotou, ich poradie môže byť ľubovoľné)  
    for each v in c[u]:  
        traverse(v)
```

Keď volanie `traverse(N-1)` skončí, operácia `collect()` **odstráni zo stroja všetky guľičky** a následne vráti pole S .

Napríklad, nech má náš stroj $N = 7$ vrcholov, pričom rodičia týchto vrcholov sú popísaní poľom $P = [4, 6, 5, 5, 6, 6]$. Na obrázku nižšie je naľavo nakreslené ako vyzerá Paulínkin stroj. Čísla v krúžkoch sú čísla jednotlivých vrcholov. Obrázok vpravo potom ukazuje, ako môže vyzeráť tento stroj po vložení nejakých guľičiek. (Presné poradie operácií `insert`, ktoré zodpovedá tomuto obrázku, nájdete v časti Ukážkový príklad.)



Ked' zavoláme funkciu `collect()`, jej beh bude vyzerat' nasledovne: Kedže koreň stroja je obsadený, inicializuje sa $S = []$ a zavolá sa `traverse(6)`.

traverse(6): vrchol 6 obsahuje guľičku s hodnotou 0, táto hodnota sa pridá do poľa S (teda $S = [0]$). Všetky deti vrchola 6, konkrétne vrcholy s číslami 4, 1 a 5, sú obsadené, ich hodnoty sú 20, 10 a 20. Utriedením týchto hodnôt dostaneme pole detí $c[6] = [1, 5, 4]$ (keďže vrcholy 4 a 5 obsahujú guľičku s rovnakou hodnotou, aj pole $c[6] = [1, 4, 5]$ by bolo prípustné). Funkcia `traverse` sa teraz rekurzívne zavolá na deti v poradí určenom $c[6]$:

- **traverse(1)**: vrchol 1 obsahuje guľičku s hodnotou 10. Tá sa pridá do S , teda $S = [0, 10]$. Keďže vrchol 1 nemá žiadne obsadené deti, toto volanie skončí.
- **traverse(5)**: vrchol 5 obsahuje guľičku s hodnotou 20. Tá sa pridá do S , teda $S = [0, 10, 20]$. Vrchol 5 má jedno obsadené dieťa s číslom 2. Teda $c[5] = [2]$ a funkcia zavolá:
 - **traverse(2)**: vrchol 2 obsahuje guľičku s hodnotou 30. Tá sa pridá do S , teda $S = [0, 10, 20, 30]$. Keďže vrchol 2 nemá žiadne obsadené deti, toto volanie skončí.
 - Keďže sme už spracovali všetky deti vrcholu 5, aj volanie `traverse(5)` následne skončí.
- **traverse(4)**: vrchol 4 obsahuje guľičku s hodnotou 20. Tá sa pridá do S , teda $S = [0, 10, 20, 30, 20]$. Keďže vrchol 4 nemá žiadne obsadené deti, toto volanie skončí.

Po skončení rekurzívnej funkcie `traverse` sa zo stroja odstránia všetky guľičky a `collect` vráti na výstup pole $S = [0, 10, 20, 30, 20]$.

Vašou úlohou je zistiť počet vrcholov N v stroji a vypočítať pole rodičov $R = [R[0], R[1], \dots, R[N-2]]$, ktoré popisuje štruktúru stroja. Tento popis môže (na rozdiel oproti pôvodnému číslovaniu, ktoré používa testovač) použiť iné číslovanie vnútorných vrcholov. Inými slovami, vrcholy iné ako listy a koreň môžete očíslovať v ľubovoľnom poradí, len následne musíte poľom R popísať správnu štruktúru stroja.

Formálne, pole R je považované za korektné, ak je možné vytvoriť prečíslovanie L , ktoré z vášho stroja vyrobí stroj používaný testovačom. Toto prečíslovanie L musí byť permutácia, teda každému vrcholu u stroja musí priradiť inú hodnotu $L[u]$ z rozsahu $0 \leq L[u] < N$. Navyše musí platiť:

- $L[u] = u$ pre všetky $0 \leq u < M$ a aj pre $u = N - 1$;
- $L[P[u]] = R[L[u]]$ pre všetky $0 \leq u < N - 1$.

Nie je vyžadované aby pre všetky u platilo $R[u] > u$.

Stroj musí byť na konci behu vášho programu **prázdny**.

Nech K je počet volaní funkcie `collect` a B najväčšia hodnota napísaná na niektorej guľičke naprieč všetkými volaniami funkcie `insert`. Nech $C = K + B$. Takto definovaná hodnota C **nesmie prekročiť** 1000. Navyše, v niektorých podúlohách bude hodnotenie za podúlohu závisieť od hodnoty C .

Implementačné detaily

Vašou úlohou je implementovať nasledovnú funkciu.

```
std::vector<int> find_structure(int M)
```

- M : počet listov stroja.
- Táto funkcia je zavolaná práve raz pre každý testovací vstup.
- Táto funkcia musí vrátiť pole $R = [R[0], R[1], \dots, R[N-2]]$ dĺžky $N-1$ popisujúce rodičov vrcholov stroja.

Vaše riešenie môže volať nasledovné dve funkcie.

```
bool insert(int U, int X)
```

- U : číslo listu, do ktorého chcete skúsiť vložiť guľičku. Musí platiť $0 \leq U < M$.
- X : celočíselná hodnota guľičky. Musí platiť $0 \leq X \leq 1000$.
- Táto funkcia vráti `true` ak bola guľička úspešne vložená do stroja, inak, ak je vrchol U obsadený, vráti hodnotu `false`.
- Túto funkciu môžete zavolať nanajvýš 500 000 krát.

```
std::vector<int> collect()
```

- Zozbiera hodnoty guľičiek v stroji podľa vyššie popísaného postupu, odstráni ich zo stroja a vráti pole S .

Ak kedykoľvek počas behu programu presiahne hodnota C hodnotu 1000, vaše riešenie dostane verdikt `Output isn't correct: Too many resources used.`

Štruktúra stroja je **fixne určená** ešte pred volaním funkcie `find_structure`. Testovač je **deterministický** v zmysle, že dva behy programu na konkrétnom vstupe, ktoré spravia tú istú postupnosť operácií budú mať rovnaké návratové hodnoty pre všetky volania funkcií `collect`.

Obmedzenia

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Podúlohy

Podúlohy	Body	Dodatočné obmedzenia
1	5	Koreň má presne M detí.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Bez dodatočných obmedzení.

V podúlohách 3 a 4 bude vaše skóre závisieť od dosiahnutej hodnoty C a to nasledovne.

Podúloha 3 (25 bodov)

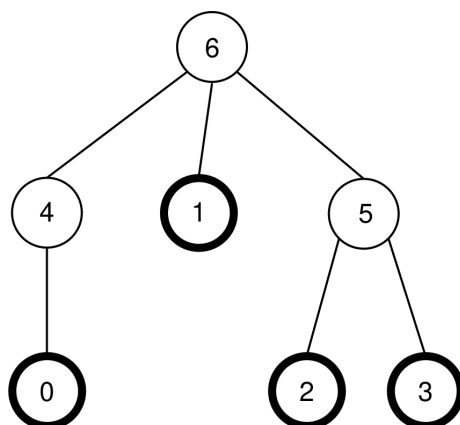
Limity	Body
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Podúloha 4 (60 bodov)

Limity	Body
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Ukážkový príklad

Nech stroj vyzerá nasledovne: $N = 7$, $M = 4$ a $P = [4, 6, 5, 5, 6, 6]$ (štruktúra zhodná s príkladom použitým vyššie).



Testovač spraví volanie

```
find_structure(4)
```

Táto funkcia následne spraví nasledovné volania:

1. **insert(0, 0)**: list 0 je voľný, guľička s hodnotou 0 je vložená do listu 0. Keďže rodič $P[0] = 4$ je voľný, guľička sa posunie do vrcholu 4. Aj vrchol $P[4] = 6$ je voľný, guľička sa posunie do vrcholu 6. Vrchol 6 je koreňom, guľička sa teda ďalej nehýbe. Toto volanie vráti `true`.
2. **insert(3, 20)**: list 3 je voľný, guľička s hodnotou 20 je vložená do listu 3. Keďže rodič $P[3] = 5$ je voľný, guľička sa posunie do vrcholu 5. Vrchol $P[5] = 6$ je obsadený, guľička sa teda ďalej nehýbe. Toto volanie vráti `true`.
3. **insert(1, 10)**: list 1 je voľný, guľička s hodnotou 10 je vložená do listu 1. Vrchol $P[1] = 6$ je obsadený, guľička sa teda ďalej nehýbe. Toto volanie vráti `true`.
4. **insert(2, 30)**: list 2 je voľný, guľička s hodnotou 30 je vložená do listu 2. Vrchol $P[2] = 5$ je obsadený, guľička sa teda ďalej nehýbe. Toto volanie vráti `true`.
5. **insert(1, 25)**: list 1 je obsadený, guľička do stroja nie je vložená a funkcia vráti `false`.
6. **insert(0, 20)**: list 0 je voľný, guľička s hodnotou 20 je vložená do listu 0. Keďže rodič $P[0] = 4$ je voľný, guľička sa posunie do vrcholu 4. Vrchol $P[4] = 6$ je obsadený, guľička sa teda ďalej nehýbe. Toto volanie vráti `true`.

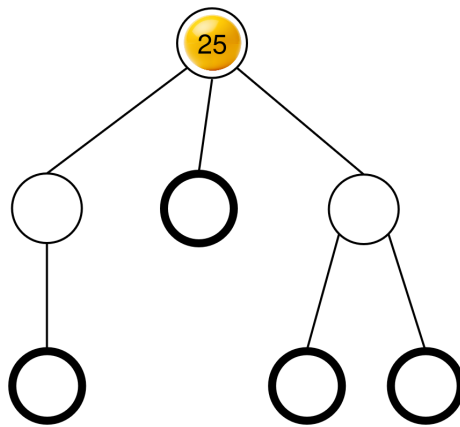
Výsledné umiestnenie guľičiek v stroji je zobrazené v pravej časti prvého obrázku v zadaní.

Následne sa zavolá

```
collect()
```

Beh tohto volania bol popísaný v zadaní. Výstupom tohto volania je pole $S = [0, 10, 20, 30, 20]$. Po tomto volaní je stroj prázdny, teda neobsahuje žiadne guľičky.

Potom sa zavolá `insert(2,25)`. List 2 je voľný, guľička s hodnotou 0 je vložená do listu 2. Keďže rodič $P[2] = 5$ je voľný, guľička sa posunie do vrcholu 5. Vrchol $P[5] = 6$ je tiež voľný, guľička sa preto presunie do vrcholu 6. Vrchol 6 je koreňom, guľička sa teda ďalej nehýbe. Toto volanie vráti `true`. Výsledný stav stroju je zobrazený na obrázku nižšie.



Na záver funkcia zavolá `collect()`, výsledkom čoho bude pole $S = [25]$ a prázdny stroj.

Existujú dve správne polia rodičov, ktoré môže `find_structure` vrátiť: $R = P = [4, 6, 5, 5, 6, 6]$ (zodpovedá prečíslovaniu $L = [0, 1, 2, 3, 4, 5, 6]$) a $R = [5, 6, 4, 4, 6, 6]$ (zodpovedá prečíslovaniu $L = [0, 1, 2, 3, 5, 4, 6]$). V tomto prípade bolo $K = 2$ (počet volaní `collect`) a $B = 30$ (najväčšia použitá hodnota guľičky), takže $C = 32$.

Ukážkový testovač

Formát vstupu:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Formát výstupu:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Q je dĺžka poľa R vrátená funkciou `find_structure`.